

Modeling and MPPI-Based Swing-Up Control of a Double Inverted Pendulum on a Cart

Author Name

Department of Mechanical and Aerospace Engineering
Course Project Report

Abstract—This paper presents a detailed modeling and control write-up for a double inverted pendulum on a cart driven by Model Predictive Path Integral (MPPI) control. The system is implemented as a six-state, single-input nonlinear cart-pole model with two serial pendulum links whose masses are concentrated at the link tips. We first derive the manipulator-form dynamics used in the implementation and clarify the state and angle conventions needed to avoid ambiguity. We then review MPPI from a sampling-based stochastic optimal control viewpoint and show how the implemented controller combines random control perturbations, forward simulation, exponential trajectory weighting, and receding-horizon replanning. Special attention is given to notation, reference generation, cost shaping, saturation handling, and track constraints, because these details determine whether the controller merely moves the cart or actually achieves repeatable swing-up and terminal balancing. The resulting controller uses a short-horizon sampling policy update around a nominal control sequence and an implementation-oriented cost function that blends position tracking, angular regulation, velocity damping, uprightness rewards, terminal penalties, and soft boundary avoidance. The write-up is intended to serve as a reproducible paper-style description of the provided code base rather than a purely conceptual overview.

Index Terms—double inverted pendulum, cart-pole, MPPI, sampling-based control, swing-up, nonlinear dynamics

I. INTRODUCTION

The double inverted pendulum on a cart is a classic underactuated benchmark because one horizontal cart force must simultaneously translate the base, inject energy into two coupled links, and stabilize the unstable upright equilibrium. Compared with the single-link cart-pole, the second link introduces stronger nonlinear coupling, multiple swing modes, and a narrower margin between successful swing-up and loss of coordination. As a result, the problem is a useful testbed for nonlinear optimal control methods that can reason over a finite horizon without depending on a local linearization alone.

Among such methods, MPPI is attractive because it replaces the solution of a nonlinear program with repeated sampling and weighted averaging of candidate control sequences. This makes the method straightforward to implement on top of a simulation model, while still allowing highly nonlinear cost shaping. The present repository follows exactly that philosophy: the controller repeatedly samples perturbed force sequences, predicts the state trajectories with the nonlinear dynamics, scores each trajectory, and updates the nominal input sequence by an importance-weighted average.

This report organizes the implementation into a paper-style narrative. The main contributions are threefold. First, the exact

dynamics used in the code are written explicitly in matrix form, including damping and inertial coupling. Second, the MPPI controller is described from both the general algorithmic viewpoint and the exact implementation viewpoint, including the specific rollout, weighting, and sequence-shifting operations. Third, the cost and constraints are documented carefully so that the manuscript can be used as a faithful technical reference for the code.

II. SYSTEM MODEL AND NOTATION

A. Coordinates and State Definition

Let the generalized coordinate vector be

$$\mathbf{q} = [x \quad \theta_1 \quad \theta_2]^\top, \quad (1)$$

where $x \in \mathbb{R}$ is the cart position, θ_1 is the angle of the first link, and θ_2 is the angle of the second link. In the implementation, both angles are measured from the upright direction. Hence,

$$\theta_1 = \theta_2 = 0 \quad (2)$$

corresponds to the upright equilibrium, while the nominal hanging configuration is near

$$\theta_1 = \theta_2 = \pi. \quad (3)$$

The full state is

$$\mathbf{x} = [x \quad \theta_1 \quad \theta_2 \quad \dot{x} \quad \dot{\theta}_1 \quad \dot{\theta}_2]^\top \in \mathbb{R}^6, \quad (4)$$

and the control input is the horizontal cart force

$$u \in \mathbb{R}. \quad (5)$$

The model parameters are the cart mass m_0 , link-tip masses m_1 and m_2 , link lengths l_1 and l_2 , gravity g , cart damping d_x , and joint damping coefficients d_1 and d_2 . The code uses

$$m_0 = 1.0, \quad m_1 = m_2 = 0.2, \quad l_1 = l_2 = 0.5, \quad (6)$$

with $g = 9.81 \text{ m/s}^2$ in the default model.

B. Geometric Convention

Each pendulum mass is modeled as a point mass concentrated at the tip of its corresponding rigid, massless link. The cart position is

$$\mathbf{p}_c = \begin{bmatrix} x \\ 0 \end{bmatrix}, \quad (7)$$

TABLE I
MAIN NOTATION

Symbol	Meaning
$\mathbf{q}$	generalized coordinates $[x, \theta_1, \theta_2]^\top$
$\mathbf{x}$	full state vector
u	cart force input
$\mathbf{x}^*$	final goal state
N	prediction horizon length
M	number of sampled trajectories
σ	input perturbation standard deviation
λ	MPPI temperature parameter
$\bar{\mathbf{u}}$	nominal input sequence
$\mathbf{x}_k^{\text{ref}}$	reference state at step k
u_k^{ref}	reference control at step k
$S^{(m)}$	rollout cost of sample m
$w^{(m)}$	normalized sample weight
η_k	uprightness score
p_k	normalized progress score

the first tip position is

$$\mathbf{p}_1 = \mathbf{p}_c + \begin{bmatrix} l_1 \sin \theta_1 \\ l_1 \cos \theta_1 \end{bmatrix}, \quad (8)$$

and the second tip position is

$$\mathbf{p}_2 = \mathbf{p}_1 + \begin{bmatrix} l_2 \sin \theta_2 \\ l_2 \cos \theta_2 \end{bmatrix}. \quad (9)$$

These expressions are used in the visualization and also in the environment-level link-span checks that prevent the links from extending past the track bounds.

C. Notation Summary

The following table summarizes the principal symbols used in the remainder of the manuscript.

D. Manipulator-Form Dynamics

The implemented dynamics can be written in the standard second-order form

$$\mathbf{M}(\mathbf{q})\ddot{\mathbf{q}} = \boldsymbol{\tau}(\mathbf{q}, \dot{\mathbf{q}}, u), \quad (10)$$

with mass matrix

$$\mathbf{M}(\mathbf{q}) = \begin{bmatrix} m_0 + m_1 + m_2 & (m_1 + m_2)l_1 \cos \theta_1 & m_2 l_2 \cos \theta_2 \\ (m_1 + m_2)l_1 \cos \theta_1 & (m_1 + m_2)l_1^2 & m_2 l_1 l_2 \cos(\theta_1 - \theta_2) \\ m_2 l_2 \cos \theta_2 & m_2 l_1 l_2 \cos(\theta_1 - \theta_2) & m_2 l_2^2 \end{bmatrix} \quad (11)$$

The generalized force vector used in the code is

$$\boldsymbol{\tau} = \begin{bmatrix} u - d_x \dot{x} + (m_1 + m_2)l_1 \sin \theta_1 \dot{\theta}_1^2 + m_2 l_2 \sin \theta_2 \dot{\theta}_2^2 \\ -d_1 \dot{\theta}_1 - m_2 l_1 l_2 \sin(\theta_1 - \theta_2) \dot{\theta}_2^2 + (m_1 + m_2)g l_1 \sin \theta_1 \\ -d_2 \dot{\theta}_2 + m_2 l_1 l_2 \sin(\theta_1 - \theta_2) \dot{\theta}_1^2 + m_2 g l_2 \sin \theta_2 \end{bmatrix} \quad (12)$$

Equation (10) is solved at every dynamics evaluation by a dense linear solve. The continuous-time first-order state equation is then

$$\dot{\mathbf{x}} = [\dot{x} \quad \dot{\theta}_1 \quad \dot{\theta}_2 \quad \ddot{x} \quad \ddot{\theta}_1 \quad \ddot{\theta}_2]^\top. \quad (13)$$

The force command is clipped before entering the dynamics,

$$u_{\text{sat}} = \text{clip}(u, -u_{\text{max}}, u_{\text{max}}), \quad (14)$$

where the demo script sets $u_{\text{max}} = 50$ N.

III. MPPI PRELIMINARIES

A. General MPPI Idea

MPPI is a sampling-based receding-horizon controller that maintains a nominal control sequence

$$\bar{\mathbf{u}} = \{\bar{u}_0, \bar{u}_1, \dots, \bar{u}_{N-1}\}, \quad (15)$$

generates noisy candidate sequences, rolls each sequence through the nonlinear system model, and computes a weighted average that shifts the nominal policy toward low-cost trajectories. Let the m th sampled perturbation sequence be

$$\boldsymbol{\epsilon}^{(m)} = \{\epsilon_0^{(m)}, \epsilon_1^{(m)}, \dots, \epsilon_{N-1}^{(m)}\}, \quad (16)$$

with

$$\epsilon_k^{(m)} \sim \mathcal{N}(0, \sigma^2). \quad (17)$$

The candidate control is

$$u_k^{(m)} = \text{clip}(\bar{u}_k + \epsilon_k^{(m)}, -u_{\text{max}}, u_{\text{max}}). \quad (18)$$

After rolling out all M candidates and obtaining trajectory costs $S^{(m)}$, MPPI forms the normalized weights

$$w^{(m)} = \frac{\exp\left(-\frac{S^{(m)} - \beta}{\lambda}\right)}{\sum_{j=1}^M \exp\left(-\frac{S^{(j)} - \beta}{\lambda}\right)}, \quad (19)$$

where $\lambda > 0$ is the temperature parameter and

$$\beta = \min_m S^{(m)} \quad (20)$$

is used for numerical stabilization. The new nominal sequence becomes

$$\bar{\mathbf{u}} \leftarrow \sum_{m=1}^M w^{(m)} \mathbf{u}^{(m)}. \quad (21)$$

The first action is applied to the real system, the sequence is shifted forward by one step, and the last element is reset to zero before the next control cycle.

B. Preliminary Remarks on Local Models

The repository also includes a finite-difference linearization routine around an equilibrium pair $(\mathbf{x}_{\text{eq}}, u_{\text{eq}})$. With perturbation size ε , the Jacobians are approximated by

$$\mathbf{A}_{:,i} \approx \frac{\mathbf{f}(\mathbf{x}_{\text{eq}} + \varepsilon \mathbf{e}_i, u_{\text{eq}}) - \mathbf{f}(\mathbf{x}_{\text{eq}} - \varepsilon \mathbf{e}_i, u_{\text{eq}})}{2\varepsilon}, \quad (22)$$

$$\mathbf{B} \approx \frac{\mathbf{f}(\mathbf{x}_{\text{eq}}, u_{\text{eq}} + \varepsilon) - \mathbf{f}(\mathbf{x}_{\text{eq}}, u_{\text{eq}} - \varepsilon)}{2\varepsilon}. \quad (23)$$

This linear model is not used by the MPPI controller itself, but it provides useful preliminary context. In particular, it clarifies why local methods can support balancing near the upright equilibrium whereas MPPI is attractive for the full swing-up problem starting from the hanging configuration.

C. Prediction Model

The implementation uses a nonlinear prediction model identical in structure to the simulation model but integrated with fourth-order Runge–Kutta (RK4) inside the controller. For a step size Δt , one prediction step is

$$\mathbf{k}_1 = \mathbf{f}(\mathbf{x}_k, u_k), \quad (24)$$

$$\mathbf{k}_2 = \mathbf{f}\left(\mathbf{x}_k + \frac{\Delta t}{2}\mathbf{k}_1, u_k\right), \quad (25)$$

$$\mathbf{k}_3 = \mathbf{f}\left(\mathbf{x}_k + \frac{\Delta t}{2}\mathbf{k}_2, u_k\right), \quad (26)$$

$$\mathbf{k}_4 = \mathbf{f}(\mathbf{x}_k + \Delta t\mathbf{k}_3, u_k), \quad (27)$$

$$\mathbf{x}_{k+1} = \mathbf{x}_k + \frac{\Delta t}{6}(\mathbf{k}_1 + 2\mathbf{k}_2 + 2\mathbf{k}_3 + \mathbf{k}_4). \quad (28)$$

The demo uses $\Delta t = 0.05$ s and prediction horizon $N = 24$.

IV. CONTROLLER FORMULATION

A. Online Reference Generation

The controller can accept an external state and control reference trajectory, but the double-pendulum MPPI demo does not provide one. Instead, an internal smooth reference is generated online. Let the current control time be t , the horizon index be k , and the goal ramp time be T_r . Define

$$r_k = \text{clip}\left(\frac{t + k\Delta t}{T_r}, 0, 1\right), \quad (29)$$

and the cubic smoothstep function

$$s(r) = 3r^2 - 2r^3. \quad (30)$$

Then the reference sequence is

$$\mathbf{x}_k^{\text{ref}} = \mathbf{x}(t) + (\mathbf{x}^* - \mathbf{x}(t))s(r_k), \quad (31)$$

$$\theta_{1,k}^{\text{ref}} = \pi + (\theta_1^* - \pi)s(r_k), \quad (32)$$

$$\theta_{2,k}^{\text{ref}} = \pi + (\theta_2^* - \pi)s(r_k), \quad (33)$$

$$\dot{\mathbf{x}}_k^{\text{ref}} = \dot{\theta}_{1,k}^{\text{ref}} = \dot{\theta}_{2,k}^{\text{ref}} = 0. \quad (34)$$

For the swing-up task,

$$\mathbf{x}^* = [3 \ 0 \ 0 \ 0 \ 0 \ 0]^\top. \quad (35)$$

Hence the reference smoothly moves the cart from its present horizontal location toward $x = 3$ m while driving both link angles from the hanging direction to the upright one over $T_r = 8$ s.

B. Sampling Policy

The MPPI controller stores a nominal input sequence $\bar{\mathbf{u}} \in \mathbb{R}^N$. At each control update:

- 1) A Gaussian perturbation matrix of size $M \times N_c$ is sampled, where M is the number of rollouts and N_c is a coarse horizon length.
- 2) If action repetition is used, each coarse action is repeated to produce a length- N candidate sequence. In the demo, the repetition factor is one, so $N_c = N$.
- 3) Each candidate sequence is clipped to the actuator bound and rolled out through the batch RK4 predictor.

- 4) The trajectory costs are computed, weights are formed with (19), and the nominal sequence is updated using (21).
- 5) The first action is applied and the updated sequence is shifted left by one index.

The implementation parameters used by the demo are

$$N = 24, \quad M = 5000, \quad \sigma = 2.0, \quad \lambda = 10.0. \quad (36)$$

The random number generator is initialized with a fixed seed of 18, which improves repeatability of qualitative behavior.

C. Trajectory Cost Used in the Code

The implemented cost is not a single textbook quadratic form. Instead, it is a shaped objective built to support swing-up, inter-link coordination, target locking, and constraint awareness. Let $\mathbf{x}_k^{(m)}$ denote the m th sampled state at step k , and let $\mathbf{x}_k^{\text{ref}}$ and u_k^{ref} be the internal reference state and control. Define the wrapped angle errors

$$e_{\theta_{1,k}}^{(m)} = \text{wrapToPi}\left(\theta_{1,k}^{(m)} - \theta_{1,k}^{\text{ref}}\right), \quad (37)$$

$$e_{\theta_{2,k}}^{(m)} = \text{wrapToPi}\left(\theta_{2,k}^{(m)} - \theta_{2,k}^{\text{ref}}\right), \quad (38)$$

the cart tracking error

$$e_{x,k}^{(m)} = x_k^{(m)} - x_k^{\text{ref}}, \quad (39)$$

and the goal-position error

$$\tilde{e}_{x,k}^{(m)} = x_k^{(m)} - x^*. \quad (40)$$

Let the rate error vector be

$$\mathbf{e}_{v,k}^{(m)} = \begin{bmatrix} \dot{x}_k^{(m)} - \dot{x}_k^{\text{ref}} & \dot{\theta}_{1,k}^{(m)} - \dot{\theta}_{1,k}^{\text{ref}} & \dot{\theta}_{2,k}^{(m)} - \dot{\theta}_{2,k}^{\text{ref}} \end{bmatrix}^\top. \quad (41)$$

Two shaping variables are also used. First, the *uprightness*

$$\eta_k^{(m)} = \cos \theta_{1,k}^{(m)} + \cos \theta_{2,k}^{(m)}, \quad (42)$$

which is largest when both links are upright. Second, a normalized progress term

$$p_k^{(m)} = \text{clip}\left(\frac{\eta_k^{(m)} + 2}{4}, 0, 1\right), \quad (43)$$

which maps the two-link orientation quality to $[0, 1]$. A third quantity, denoted ρ_k , measures how far the cart reference has advanced toward the goal along the horizon.

With these definitions, the stage cost used in the implementation can be summarized as

$$\begin{aligned} \ell_k^{(m)} = & \left(0.10 + 2.4\rho_k + 1.8p_k^{(m)}\right) \left(e_{x,k}^{(m)}\right)^2 + 24 \left(e_{\theta_{1,k}}^{(m)}\right)^2 + 24 \left(e_{\theta_{2,k}}^{(m)}\right)^2 \\ & + 0.18 \left(e_{x,k}^{(m)}\right)^2 + 0.14 \left(e_{\theta_{1,k}}^{(m)}\right)^2 + 0.14 \left(e_{\theta_{2,k}}^{(m)}\right)^2 \\ & + 0.0022 \left(u_k^{(m)} - u_k^{\text{ref}}\right)^2 - 10\eta_k^{(m)} - 3p_k^{(m)} \cos\left(e_{\theta_{1,k}}^{(m)} - e_{\theta_{2,k}}^{(m)}\right) \\ & + 100p_k^{(m)} \left(\tilde{e}_{x,k}^{(m)}\right)^2 + \ell_{k,\text{bound}}^{(m)}. \end{aligned} \quad (44)$$

Equation (44) reveals several design choices:

- 1) The cart tracking weight grows as the motion progresses, so cart placement matters more once the links have gained uprightness.
- 2) Large angular penalties ensure that sampled trajectories align with the reference swing-up path.
- 3) The negative uprightness reward encourages energy injection toward the upward configuration even before the terminal cost dominates.
- 4) The coordination reward $-3p_k \cos(e_{\theta_1} - e_{\theta_2})$ favors coherent link motion near the upright region.
- 5) The goal-position term $100p_k \tilde{e}_x^2$ activates more strongly when the pendulum is already approaching uprightness.

D. Boundary Penalties and Terminal Cost

When cart-position bounds are active, the controller adds a soft penalty if the predicted cart gets closer than 0.28 m to either boundary:

$$\ell_{k,\text{bound}}^{(m)} = 160 \max(0, 0.28 - (x_k^{(m)} - x_{\min}))^2 + 160 \max(0, 0.28 - (x_{\max} - x_k^{(m)}))^2. \quad (45)$$

In addition, the prediction code clips the cart state itself to $[x_{\min}, x_{\max}]$ after each RK4 step.

The terminal part of the cost is even more aggressive. Let the terminal state be $x_N^{(m)}$ and let the last few predicted states define a short tail segment. The implementation penalizes both the terminal state and the last six predicted steps:

$$\begin{aligned} \phi^{(m)} = & 180(x_N^{(m)} - x_N^{\text{ref}})^2 + 420(e_{\theta_{1,N}}^{(m)})^2 + 420(e_{\theta_{2,N}}^{(m)})^2 \\ & + 18(\dot{x}_N^{(m)} - \dot{x}_N^{\text{ref}})^2 + 12(\dot{\theta}_{1,N}^{(m)} - \dot{\theta}_{1,N}^{\text{ref}})^2 \\ & + 12(\dot{\theta}_{2,N}^{(m)} - \dot{\theta}_{2,N}^{\text{ref}})^2 + 400(x_N^{(m)} - x^*)^2 \\ & + \sum_{j=N-5}^N \left[30(x_j^{(m)} - x^*)^2 + 90\bar{e}_{\theta_{1,j}}^2 + 90\bar{e}_{\theta_{2,j}}^2 \right. \\ & \left. + 4\dot{x}_j^2 + 3\dot{\theta}_{1,j}^2 + 3\dot{\theta}_{2,j}^2 \right], \end{aligned} \quad (46)$$

where $\bar{e}_{\theta_{i,j}}$ are the wrapped errors with respect to the final goal angles rather than the intermediate reference. This tail penalty makes the end of the horizon look like the beginning of a balancing phase instead of merely the end of a swing-up arc.

E. Hard Constraints in Simulation

The simulated environment imposes additional non-smooth safeguards beyond the MPPI cost:

- 1) The real control command is clipped to $\pm u_{\max}$ before integration.
- 2) The cart state is clipped if the cart itself exceeds the track bounds.
- 3) If link-span enforcement is enabled and the links would extend beyond the allowed track despite the cart staying inside, the simulator rejects that step by restoring the previous state and zeroing the velocity components.

This hybrid handling is practical: MPPI sees a softened version of the boundary through the prediction cost, while the simulator still guarantees physically admissible track occupancy.

TABLE II
MAIN MODELING AND MPPI PARAMETERS

Quantity	Value
State dimension	6
Input dimension	1
m_0	1.0 kg
m_1, m_2	0.2 kg, 0.2 kg
l_1, l_2	0.5 m, 0.5 m
g	9.81 m/s ²
d_x	0.0
d_1, d_2	0.03, 0.03
$u_{\max}$	50 N
Δt	0.05 s
Horizon N	24
Samples M	5000
Noise std. σ	2.0
Temperature λ	10.0
Goal ramp time T_r	8.0 s
Track bounds	$[-1.5, 7.5]$ m

V. IMPLEMENTATION DETAILS

A. Simulation and Prediction Pipelines

The actual closed-loop simulation uses `solve_ivp` with tight tolerances (10^{-7} relative and 10^{-9} absolute) between control updates. By contrast, the controller uses explicit RK4. This split is beneficial because it keeps the closed-loop simulation accurate while preserving a fast, vectorized predictor for MPPI.

The controller stores both the weighted nominal predicted trajectory and a stride-subsampled set of sampled trajectories. These are not needed for control, but they are useful for visualizing what the controller is considering at each time step. The implementation therefore supports diagnostic plots and animation overlays without changing the control loop itself.

B. Task Setup

The double-pendulum MPPI demo uses the following settings:

- Initial state: hanging configuration $x_0 = [0, \pi, \pi, 0, 0, 0]^T$ with optional small angle perturbation.
- Goal state: upright at a displaced cart position, $x^* = [3, 0, 0, 0, 0, 0]^T$.
- Simulation step: $\Delta t = 0.05$ s.
- Track bounds: $x \in [-1.5, 7.5]$ m.
- Damping used in the demo: $d_x = 0, d_1 = d_2 = 0.03$.
- Force limit: $u_{\max} = 50$ N.

The script also places two rectangular obstacles near $x = 1.4$ m for visualization. However, these obstacles are not currently included in the MPPI objective or dynamics constraints. They should therefore be interpreted as scene annotations rather than active collision constraints.

VI. REPRESENTATIVE CLOSED-LOOP BEHAVIOR

In a successful run, the controller first drives the cart to create kinetic energy and phase separation between the two links. During this stage, the negative uprightness term and moderate position shaping allow exploratory motion without

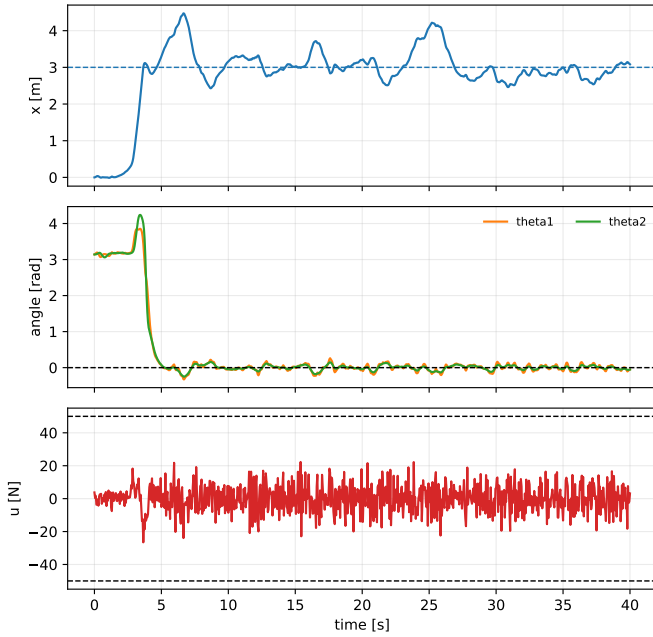


Fig. 1. Representative closed-loop response for the implemented MPPI controller. The cart approaches the target position while both links are swung up from the hanging configuration and regulated near the upright equilibrium.

demanding immediate cart placement at the final goal. As the links approach the upward region, the progress-dependent weights increase the cost of cart-position error and the terminal penalties begin to dominate. This naturally transitions the motion from energy pumping to capture and balancing.

The qualitative behavior produced by the cost design is worth emphasizing. A purely quadratic state penalty around the upright state often struggles from the hanging initial condition because it lacks any explicit incentive to seek high-energy intermediate states. Here, the terms $-\eta_k$ and $-p_k \cos(e_{\theta_1} - e_{\theta_2})$ play that role indirectly by rewarding more upright and coordinated configurations, while the tail and terminal penalties sharpen the landing near the final target.

Using the exact demo configuration from the script (40 s simulation, 5000 sampled rollouts per control update, and zero initial angle offset), the final state was

$$\mathbf{x}(T) \approx [3.0832 \quad -0.0508 \quad -0.0532 \quad -0.4094 \quad 0.3132 \quad 0.0829]^\top \quad (47)$$

relative to the goal $[3, 0, 0, 0, 0, 0]^\top$. The peak control magnitude in that run was approximately 26.55 N, well below the actuator limit of 50 N. Over the final two seconds, the mean absolute cart-position error was about 0.106 m, while the mean absolute angular errors were about 0.050 rad and 0.035 rad for the first and second links. These values are consistent with successful swing-up followed by practical stabilization near the goal.

Because MPPI replans at every step, the controller also exhibits a desirable robustness to moderate prediction mismatch and local disturbances. The nominal sequence is not treated as an open-loop commitment; instead, it is only a proposal that

is reweighted and shifted after every new state measurement. For a nonlinear underactuated system, this receding-horizon feedback effect is one of the main reasons MPPI remains effective even when the cost function is heavily shaped rather than derived from a strict optimality proof.

VII. DISCUSSION

Several aspects of the implementation are especially important from a controls perspective.

First, the state and angle convention must be documented carefully. Since $\theta_1 = \theta_2 = 0$ denotes the upright state rather than the downward one, gravity terms appear with $\sin \theta_i$ in a way that may look unusual if the reader assumes the more common downward-angle convention. The manuscript therefore keeps the upright reference explicit throughout.

Second, the controller uses a deliberately non-quadratic objective. This is not a weakness but a feature of the MPPI framework: once the prediction model is available, one can encode task knowledge directly through rewards and penalties that would be cumbersome in Riccati-based or linear MPC formulations. The progress-gated cart penalties are a good example. They prevent the controller from over-prioritizing cart placement before the pendulum has gained sufficient uprightness.

Third, the current setup does not yet solve obstacle avoidance, despite showing obstacles in the animation. Extending the cost with collision penalties based on the cart and tip positions would be a natural next step. Because the kinematics are already available, one could penalize the signed distance between the link tips and obstacle boundaries inside the sampled rollout cost without changing the overall MPPI structure.

Fourth, the code base already contains nonlinear trajectory optimization and gain-scheduled LQR modules. This opens a useful research direction: an optimized swing-up reference could be supplied to MPPI as a richer nominal trajectory, while MPPI would remain responsible for online disturbance rejection and constraint handling.

VIII. CONCLUSION

This paper-style report documented the double inverted pendulum cart model and the MPPI controller used in the provided implementation. The main system is a six-state, single-input nonlinear underactuated mechanism with point-mass links, explicit inertial coupling, and damping. The control method is a receding-horizon sampling controller that perturbs a nominal force sequence, predicts trajectories with RK4, evaluates a shaped swing-up-and-balance cost, and updates the nominal sequence by importance-weighted averaging.

The most important practical message is that successful control depends as much on the cost architecture and reference shaping as on the MPPI update itself. In the present implementation, uprightness rewards, coordination terms, progress-dependent cart penalties, soft boundary costs, and strong tail penalties work together to transform a generic sampling controller into a usable swing-up regulator for the double inverted pendulum. As a result, the manuscript can serve as

a reproducible technical companion to the code and as a starting point for future extensions such as obstacle-aware MPPI, learned proposal distributions, or hybrid trajectory-optimization-plus-MPPI control.

REFERENCES

- [1] G. Williams, A. Aldrich, and E. A. Theodorou, “Model predictive path integral control: From theory to parallel computation,” *Journal of Guidance, Control, and Dynamics*, vol. 40, no. 2, pp. 344–357, 2017.
- [2] E. Theodorou, J. Buchli, and S. Schaal, “A generalized path integral control approach to reinforcement learning,” *Journal of Machine Learning Research*, vol. 11, pp. 3137–3181, 2010.
- [3] T. Li and E. A. Theodorou, “An information theoretic approach to model predictive control,” in *Proc. IEEE Conf. Decision and Control*, 2019, pp. 5064–5071.